SelfCal: Datasets & Environment Summary

Cập nhật 2026-10-28. Thay cho `hmm_datasets_environment_summary.pdf` và `environment_summary_updated.pdf` cũ (thời CCBF/CRSG). Hướng nghiên cứu hiện tại là SelfCal-AA; trạng thái đọc ở `handoff/03_HANDOFF_NEW_CHAT.md`.

1. Dataset quan trọng nhất: descriptor dumps (PUBLIC)

`nhatlong1103/vpr-descriptor-dumps`, link https://www.kaggle.com/datasets/nhatlong1103/vpr-descriptor-dumps (public, 4.4 GB, giấy phép CC BY-NC-SA 4.0).

Đây là vector đặc trưng (global descriptor, float16, đã chuẩn hoá L2) của **mọi ảnh map và query** trong 4 benchmark, tính bằng 6 mạng VPR, kèm pose, điều kiện và tên ảnh. **Mọi thí nghiệm SelfCal chỉ cần dataset này.** Không cần ảnh gốc, không cần chạy lại mạng nơ-ron.

Khi thêm acc Kaggle mới: KHÔNG upload lại. Chỉ cần thêm dataset này vào kernel:

- kernel-metadata: `"dataset_sources": ["nhatlong1103/vpr-descriptor-dumps", "<acc>/dive-vpr-analysis"]`
- Hoặc trên giao diện notebook: **Add Input → Datasets → tìm "vpr-descriptor-dumps"**.
- Khi mount, nó nằm ở `/kaggle/input/datasets/nhatlong1103/vpr-descriptor-dumps/dump/` (hoặc `/kaggle/input/vpr-descriptor-dumps/dump/`). Runner của `automation/make_eval_kernel.py` tự tìm thư mục `meta/`.

Cấu trúc thư mục

```
dump/
  salad/     (8448-d)  SALAD: DINOv2-B + optimal-transport aggregation
  megaloc/   (8448-d)  MegaLoc
  mixvpr/    (4096-d)  MixVPR, ResNet-50
  netvlad/   (4096-d)  NetVLAD, VGG-16 + PCA-whitening 4096
  cosplace/  (2048-d)  CosPlace, ResNet-50
  dinov2/    (1536-d)  DINOv2-B: CLS + mean patch token
  meta/                pose / condition / image names
README.md
```

Mỗi thư mục descriptor có cùng 32 file. Thứ tự hàng trùng với file `meta` tương ứng.

File	Dataset	Số ảnh (map / query)
`rc_ref.npy`, `rc_q.npy`	RobotCar-Seasons v2: reference map chính thức + query train công khai (9 điều kiện)	6,954 / 1,906
`cmu_s<N>_{db,q}.npy`	Extended-CMU-Seasons, 11 slice chính thức N ∈ {6, 7, 8, 10, 11, 12, 13, 21, 22, 23, 24} (gộp thành một bản đồ thành phố 2,418 ảnh)	ví dụ s6: 272 / 1,463
`nord_<season>.npy` (spring, summer, fall, winter)	Nordland, 4 mùa căn khung 1:1 (subsample)	4,500 mỗi mùa
`msls_{cph,sf}_{db,q}.npy`	MSLS val: Copenhagen / San Francisco (bỏ panorama)	12,556 / 6,559 và 6,315 / 4,525

File trong `meta/`:

File	Khoá
`rc.npz`	`ref_names, ref_C` (tâm camera), `ref_R, ref_focal, q_names, q_C, q_R, q_cond`. Điều kiện: dawn, dusk, night, night-rain, overcast-summer, overcast-winter, rain, snow, sun
`cmu_s*.npz`	`db_names, db_C, db_R, q_names, q_C, q_R, q_cond` (ngày chụp = điều kiện)

File	Khoá
`nord.npz`	`frame_ids, seasons`
`msls_{cph,sf}.npz`	`db_keys, db_C` (UTM east/north), `db_ca` (compass), `db_view, db_date, q_keys, q_C, q_ca, q_view, q_date, q_cond` (city-season, cph-night)

Loader: `analysis/vprlib.py`, dùng `V.LOADERS[ds](dump_dir, descriptor)` với `ds` ∈ `rc, cmu, nord, msls, nordw` (`nordw` = Nordland lấy mùa đông làm map).

Nguồn gốc dump: do các kernel GPU của acc `nhatlong1103` tạo ra: `dive-vpr-nb15a/b/c/d-dump` (RobotCar, CMU, Nordland) và `dive-vpr-nb16a/b-msls` (MSLS). Notebook nằm ở `nb15_src/`. Chỉ cần chạy lại các kernel này khi muốn thêm descriptor hoặc dataset mới.

2. Datasets ảnh gốc và trọng số (chỉ cần khi tạo dump mới)

Slug	Nội dung
`nhatlong1103/robotcar-seasons-v2`	RobotCar-Seasons v2: ảnh, `robotcar_v2_train.txt` (pose 4×4 phẳng, translation = index [3,7,11])
`nhatlong1103/extended-cmu-seasons` (+ `-2`)	Extended-CMU-Seasons, slice 2–25. Cấu trúc lồng `sliceN/sliceN/`; pose `camera-poses/*.txt`, xyz = 3 số cuối
`nhatlong1103/nordland-full-47-6gb`	Nordland đủ 4 mùa (35,768 ảnh/mùa, căn khung 1:1)
`ralphsitinh/msls-dataset`	Mapillary SLS (public, của người khác); dùng phần val CPH + SF
`nhatlong1103/4seasons` + `nhatlong1103/4seasons-poses`	4Seasons (ảnh + pose 30 recording). Chưa dùng; dự phòng cho demo SLAM/loop-closure
`nhatlong1103/bundle`	Asset offline: `dino_salad.ckpt`, `dinov2_vitb14_pretrain.pth`, source salad/dinov2, wheels
`nhatlong1103/vpr-descriptor-weights`	Trọng số MixVPR, CosPlace, NetVLAD, MegaLoc (`automation/vpr_extra.py`)

BoQ vẫn chưa có trọng số (GitHub bị chặn, không có mirror).

3. Tài khoản Kaggle (8 acc × 2 T4 = 16 GPU chạy song song)

Token của mỗi acc nằm ở `~/.kaggle/tok_<username>` (chmod 600). `~/.kaggle/access_token` là của acc 1. Các script `automation/make_eval_kernel.py`, `run_kernel.py` và `fanout.py` tự đọc đúng token theo tên acc, nên không cần `export`. Không ghi token vào `~/.bashrc`.

#	Username	Ghi chú
1	`nhatlong1103`	Chủ dump public và dataset ảnh gốc
2	`yuan1103`	
3	`synchan5704`	
4	`loinguyentran`	
5	`yeshenhi`	
6	`mucnokieu`	
7	`huyenco`	
8	`nguynduylm`	

Cả 8 acc đã được thử chạy GPU T4 thành công (2026-10-28).

Quy tắc: - Chia job cho cả 16 slot GPU trước. - Chỉ dùng kernel CPU khi hết GPU và job đủ nhẹ. - Thêm acc mới: lưu token vào `~/.kaggle/tok_<username>`. `fanout.py` tự nhận acc đó.

4. Chạy thí nghiệm trên Kaggle

Cách nhanh nhất là chia tự động ra mọi acc và mọi slot GPU, rồi tự gom kết quả vào `analysis/results/`:

```
python3 automation/fanout.py <work_dir> <slug_prefix> "eval_all.py rc salad;eval_all.py cmu salad;..." [n_ker
```

Chạy thủ công một kernel:

```
python3 automation/make_eval_kernel.py <kernel_dir> <slug> "eval_all.py rc salad;eval_all.py cmu salad" "" --
python3 automation/run_kernel.py <kernel_dir> <out_dir> 600 3 14400
cp <out_dir>/results/*.json analysis/results/
```

- Acc khác: `--owner <username>` (token tự đọc từ `~/.kaggle/tok_<username>`). Kernel tự mount dump public, không cần upload gì thêm.
- `make_eval_kernel.py` upload `analysis/*.py` thành dataset code `<acc>/dive-vpr-analysis`, rồi viết một kernel chạy song song các job. `--gpu` bật backend PyTorch (`SELFCAL_DEVICE=cuda`).
- Trên T4, `eval_all.py` mất khoảng 0.5–1 phút cho RobotCar/CMU/Nordland và lâu hơn cho MSLS.
- Kết quả: `results/*.json`, `logs/`, `figs/` (nếu có) trong output của kernel.
- Ngắt một kernel: chỉ `kaggle kernels delete -y <owner>/<slug>` mới có tác dụng.
- Chi tiết vận hành: `handoff/02_ENGINEERING_OPS_NOTES.md`.

5. Môi trường container (Claude Code cloud)

- Không có GPU, 4 nhân CPU. Chỉ dùng local CPU cho kiểm tra nhanh (dưới ~2 phút).
- Mọi tính toán nặng chạy trên Kaggle.
- Không chạy code bên thứ ba tải về trong container; chạy nó trong kernel Kaggle.